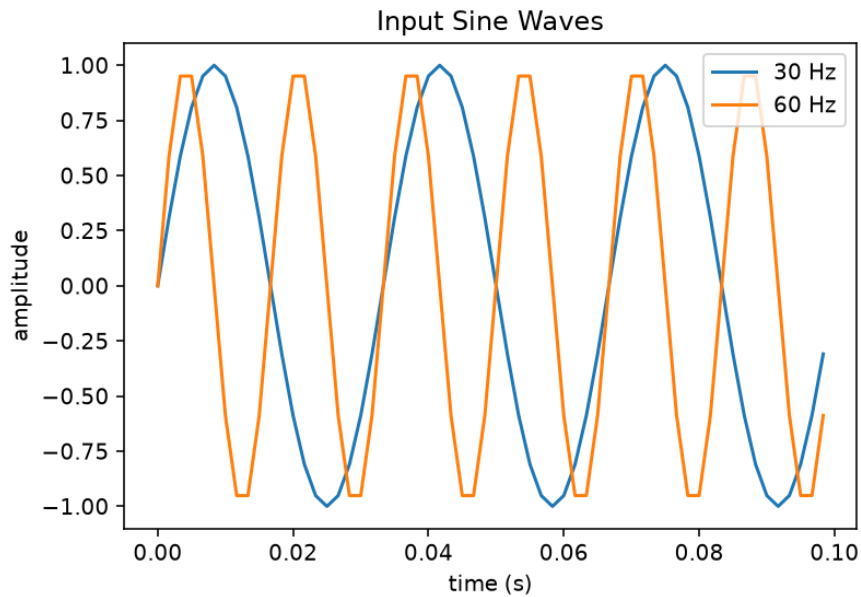


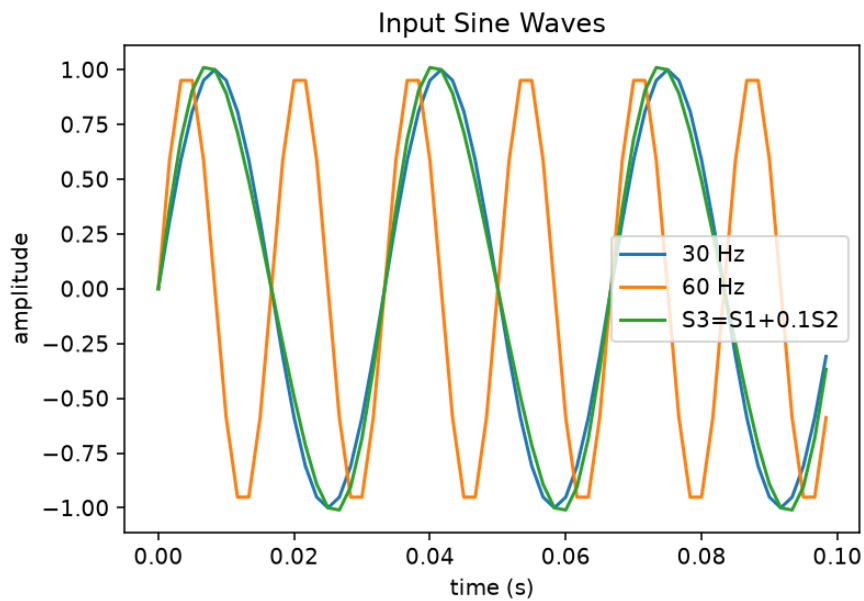
Assignment 3: FFT

1) Two sinusoidal signals were generated with a sampling frequency of 600 Hz (a 30 Hz signal and a 60 Hz signal).



2) A third signal was created:

$$S_3 = S_1 + 0.1S_2$$



3) Now, projecting signal S_3 onto signals S_1 and S_2 , we obtain:

$$S_3 \cdot S_1 = 30 \text{ and } S_3 \cdot S_2 = 3$$

```
projS3_S1
np.float64(29.999999999999996)

projS3_S2
np.float64(3.0000000000000004)
```

4) Now, let's construct a different vector $S_4 = 2S_1 - 0.5S_2$, then the projections onto the basis vectors are:

$$S_4 \cdot S_1 = 60 \text{ and } S_4 \cdot S_2 = -15$$

```
projS4_S1
np.float64(60.0)

projS4_S2
np.float64(-15.0)
```

5) By analyzing the values obtained for the projections of the vectors from exercises 3 and 4, we can observe that the projections are equivalent to the contribution of each signal to those generated as a linear combination. For example, signal S_3 consists of one instance of S_1 and 0.1 times S_2 , and since the number of samples for both signals is 60 ($600 \times 0.1 = 60$), and since they have unit amplitude and contain an integer number of cycles, their dot products with themselves are (given that the average value of a squared sine function is $\frac{1}{2}$):

$$\begin{cases} S_i \cdot S_j = 30 & \text{if } i = j \\ S_i \cdot S_j = 0 & \text{if } i \neq j \end{cases}$$

So, the projection of S_3 onto S_1 would be:

$$S_3 \cdot S_1 = (S_1 + 0.1S_2) \cdot S_1 = (S_1 \cdot S_1) + 0.1(S_2 \cdot S_1) = 30 + 0.1(0) = 30$$

and its projection onto S_2

$$S_3 \cdot S_2 = (S_1 + 0.1S_2) \cdot S_2 = (S_1 \cdot S_2) + 0.1(S_2 \cdot S_2) = 0 + 0.1(30) = 3$$

And likewise for vector $S_4 = 2S_1 - 0.5S_2$, with its projections 60 and -15 onto S_1 and S_2 , respectively.

Finally, if we normalize our basis vectors, we would obtain the same coefficients for the linear combination.

6) The obtained Fourier transform was shifted so that the center frequency was $f=0$.

Autograder Results
Results
Code

test_fft_amp(test_week3.TestWeek3) (1/1)
test_fft_f(test_week3.TestWeek3) (1/1)
test_fft_ft(test_week3.TestWeek3) (1/1)
test_fft_phase(test_week3.TestWeek3) (1/1)
test_mydft(test_week3.TestWeek3) (0/2)
Test Failed: name 'myslowdft' is not defined
test_mydft_size(test_week3.TestWeek3) (0/1)
Test Failed: name 'myslowdft' is not defined

Assignment 3 -- Fourier Transforms 1
Ungraded

Student
Holger Quintero
View or edit group

Total Points
- / 7 pts

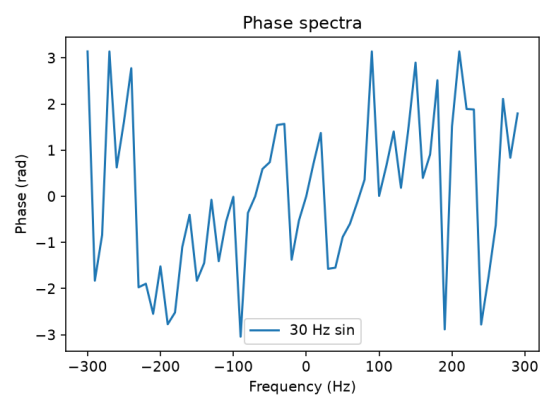
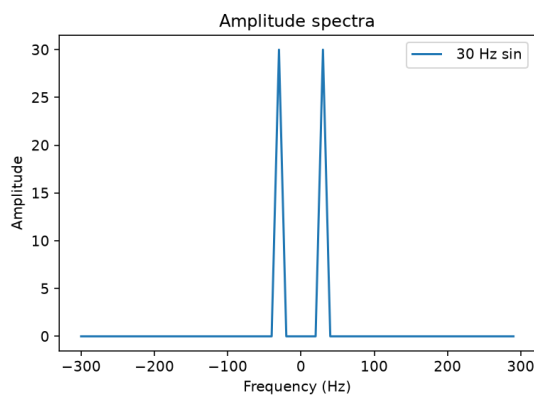
Autograder Score
4.0 / 7.0

Failed Tests
test_mydft(test_week3.TestWeek3) (0/2)
test_mydft_size(test_week3.TestWeek3) (0/1)

Passed Tests
test_fft_amp(test_week3.TestWeek3) (1/1)
test_fft_f(test_week3.TestWeek3) (1/1)
test_fft_ft(test_week3.TestWeek3) (1/1)
test_fft_phase(test_week3.TestWeek3) (1/1)

Select a question.
Group Members
Submission History
Download Submission
Resubmit

7) The amplitude and phase spectra of the Fourier transform of a 30 Hz sinusoidal signal are presented below.



8) a) dt – the time sampling interval

$$dt = \frac{1}{f_s} = \frac{1}{600} = 0.0016 \text{ s}$$

```
dt=t[2]-t[1]
dt
np.float64(0.0016666666666666668)
```

b) df – the frequency sampling interval

$$df = \frac{f_s}{N} = \frac{600}{60} = 10 \text{ Hz}$$

```
df=mysign.f[2]-mysign.f[1]
df
np.float64(10.0)
```

(c) nt – the number of time points

$$nt = 60$$

```
nt=np.size(t)
nt
60
```

(d) nf – the number of frequency points

$$nf = 60$$

```
nf=np.size(mysign.f)
nf
60
```

(e) t_{max} – the maximum time of the signal

$$t_{max} = 0.1 - dt = 0.1 - 0.0016 = 0.098 \text{ s}$$

```
tmax=np.max(t)
tmax
np.float64(0.09833333333333334)
```

(f) f_{max} – the maximum frequency of the signal

$$f_{max} = 300 - df = 300 - 10 = 290 \text{ Hz}$$

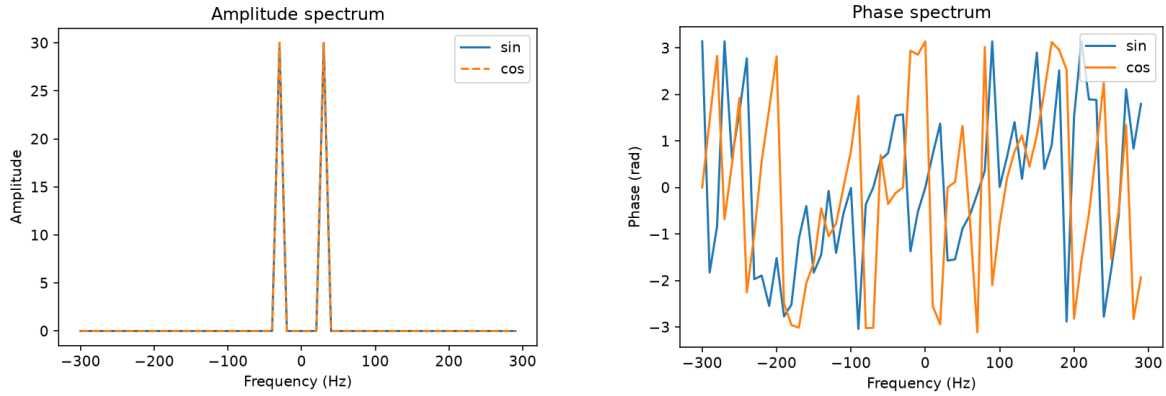
```
fmax=np.max(mysign.f)
fmax
np.float64(290.0)
```

(g) Calculate f_N – the Nyquist frequency for your dt

$$F_N = \frac{1}{2dt} = \frac{1}{2 * 0.0016} \approx 300 \text{ Hz}$$

```
nyquist_f=1/(2*dt)
nyquist_f
np.float64(300.0)
```

9) Here, I decided to consider the cosine signal $S = \cos(2\pi(30)t)$ because it yields the same amplitude spectrum, given that it shares the same frequency as the sine signal. It also allows us to obtain a different phase spectrum, as the two signals are phase-shifted relative to each other.



Here, too, we can observe that although the Fourier transforms of the two signals will not be identical (in fact, the real and imaginary parts will be swapped), their amplitude spectra will be the same, because the amplitude spectrum is calculated using the magnitude (or norm) of each complex number.

$$|S_{FT}| = \sqrt{Real(S_{FT})^2 + Imag(S_{FT})^2}$$

10) The Nyquist frequency is $F_N = 300 \text{ Hz}$ and the maximum positive frequency is $F_{max} = 290 \text{ Hz}$; this occurs because there are an even number of samples (60), and since the Fourier transform's shift function shifts the spectrum to $-F_N$ by definition, it does not reach the positive F_N , making F_{max} the highest frequency attained.

11) The relationship between d_f and the maximum signal time (T) is inverse; that is:

$$d_f = \frac{1}{T}$$

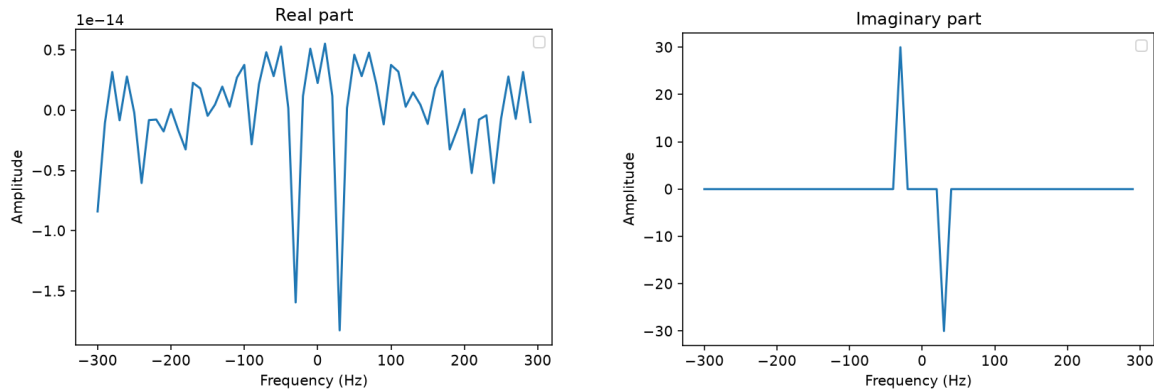
However, $t_{max} = 0.098 \text{ s}$, as this is the final time value generated by the time vector function; therefore, in this case, it is appropriate to:

$$d_f = \frac{1}{t_{max} + dt} = \frac{1}{0.1} = 10 \text{ Hz}$$

12) We can see that the peaks in the signal's amplitude spectrum occur at -30 Hz and 30 Hz; this is due to the frequency of the original signal. Since the Fourier transform is symmetric in the frequency domain, these two peaks appear, yet both represent the

frequency of the input signal. Similarly, while negative frequencies lack physical meaning, they arise from the mathematical representation of complex numbers.

13) We can observe that the phase spectrum of the signal's Fourier transform exhibits numerous peaks that might appear to be an error. It is worth noting, however, that this does not mean the signal contains many values in the frequency domain.

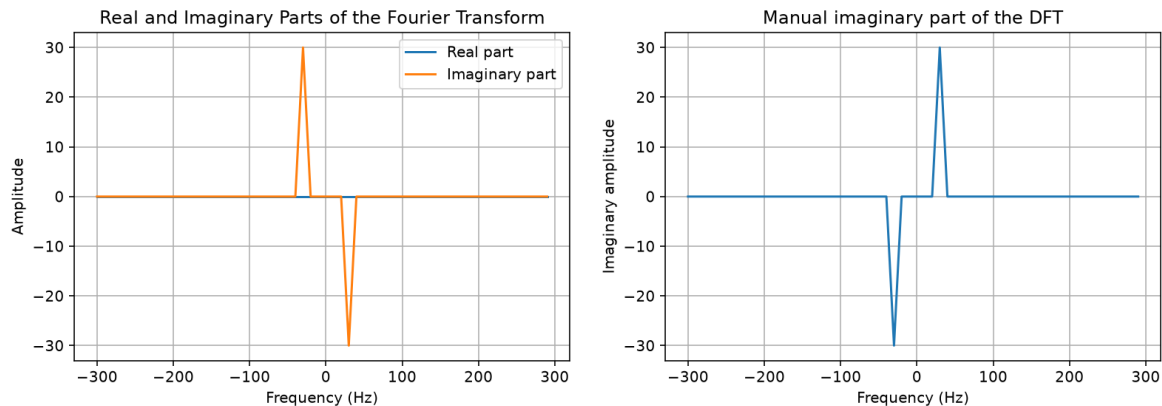


The phase is calculated as follows:

$$\varphi = \tan^{-1} \left(\frac{\text{Imag}(S_{FT})}{\text{Real}(S_{FT})} \right)$$

Since the real part of the signal is not perfectly zero, performing this operation involves values that while seemingly insignificant they are actually significant for the calculation; this is why the resulting graph shows so many peaks.

14) The imaginary part of the Fourier transform was calculated by projecting the signal onto each frequency. We can see here that if we want it to match the imaginary part of the Fast Fourier Transform exactly, we would need to multiply our manually computed transform by -1, This difference results from the sign convention used in the Fourier Transform. However, doing so would cause our Gradescope test to fail.



Without the negative sign, our function passed the Gradescope test:

Autograder Results

Results

Code

test_fft_amp (test_week3.TestWeek3) (1/1)

test_fft_f (test_week3.TestWeek3) (1/1)

test_fft_ft (test_week3.TestWeek3) (1/1)

test_fft_phase (test_week3.TestWeek3) (1/1)

test_mydft (test_week3.TestWeek3) (2/2)

test_mydft_size (test_week3.TestWeek3) (1/1)

Assignment 3 -- Fourier Transforms 1

● Ungraded

Student
Holger Quintero
[View or edit group](#)

Total Points
- / 7 pts

Autograder Score
7.0 / 7.0

Passed Tests
test_fft_amp (test_week3.TestWeek3) (1/1)
test_fft_f (test_week3.TestWeek3) (1/1)
test_fft_ft (test_week3.TestWeek3) (1/1)
test_fft_phase (test_week3.TestWeek3) (1/1)
test_mydft (test_week3.TestWeek3) (2/2)
test_mydft_size (test_week3.TestWeek3) (1/1)

Select a question.

Group Members

Submission History

Download Submission

Resubmit

15) Three different tests were conducted using varying sample sizes (500, 3,000, and 10,000), and it was observed that as the number of samples increased, so did the difference in execution times between the implementations. We can conclude that using the fast transform is more efficient than using the manual calculation of the Discrete Fourier Transform

```
Number of samples: 500
Manual DFT: 0.015028238296508789 seconds
NumPy FFT: 0.0005903244018554688 seconds
FFT is approximately 25.457592891760903 times faster
```

```
Number of samples: 3000
Manual DFT: 0.2227928638458252 seconds
NumPy FFT: 0.0026721954345703125 seconds
FFT is approximately 83.37446466809422 times faster
```

```
Number of samples: 10000
Manual DFT: 1.6221606731414795 seconds
NumPy FFT: 0.01559305191040039 seconds
FFT is approximately 104.03099293599584 times faster
```